

Zigzag Route Planning and Semantic Georouting for Agricultural Navigation

Luis Prieto-Lopez, Sergio Sanchez de La Fuente, Miguel A. Gonzalez-Santamarta
Angel Manuel Guerrero-Higueras, Lidia Sanchez-Gonzalez, Francisco J. Rodriguez-Lera

University of Leon, Leon, Spain
luisprietolopez@protonmail.com

Abstract—Agricultural robot missions often combine two planning operations that are prepared with different tools: dense coverage inside a selected plot and sparse navigation between outdoor targets. This separation creates friction when operators must verify geographic coordinates, tune route density, compare path alternatives, and export files for downstream robot software. This paper presents GeoZigzag Studio, a web-based prototype that integrates field coverage planning, semantic mission routing, optional OpenStreetMap/OSRM routing, manual forbidden zones, status-coded feedback, collapsible route details, and CSV/YAML waypoint export. The implementation uses a Leaflet browser interface and a dependency-light Python core for reproducible route generation. The current system is evaluated as a software artifact: a command-line demo regenerates 224 coverage waypoints over 16 rows and semantic mission routes with 54 and 83 waypoints, while browser screenshots show the latest route-preparation workflow. The contribution is a publication-ready integration for preparing and inspecting agricultural waypoint missions, not a claim of autonomous field validation.

Index Terms—agricultural robotics, coverage planning, waypoint navigation, geospatial planning, OpenStreetMap, ROS2

I. INTRODUCTION

Agricultural mobile robots are increasingly expected to operate across several spatial scales. At the plot scale, the robot may need to follow a dense boustrophedon or zigzag trajectory to inspect crops, apply treatment, or collect sensor data. At the farm or site scale, the same robot may need to move between water points, tracks, crop zones, or semantic targets represented in a geographic dataset. These two planning tasks are related, but they are commonly prepared with different tools and mental models.

This separation is inconvenient in experimental robotics. A coverage planner usually exposes row spacing, sweep direction, and waypoint spacing, whereas a geographic route planner exposes target order, map layers, road or path routing, and obstacle information. Moving from one tool to the other forces the operator to re-check WGS84 coordinate order, map context, route density, and export formats. In ROS2-based workflows, another practical issue appears: both routes eventually need to become waypoint files with heading information that a downstream waypoint follower can consume.

The problem also affects scientific reporting. When screenshots, metrics, route files, and code live in different places, a paper can easily describe a feature that no longer exists or show an interface that no longer matches the implementation. A route-generation tool used in a paper should therefore be

traceable: the same repository should contain the web interface, deterministic route-generation scripts, generated figures, and tests used to verify the reported behavior.

GeoZigzag Studio addresses this integration problem. The current implementation combines a static Leaflet web interface with a small Python planning package. The web app supports coverage planning, semantic mission routing, optional OpenStreetMap/OSRM routing, manual forbidden-zone polygons, a centered waypoint manager, dynamic map legends, route summaries, route-detail logs, and CSV/YAML exports. The Python package provides deterministic coverage and semantic route generation for reproducibility. Navigation launch, Gazebo simulation, and physical robot execution are intentionally outside the current paper scope.

The implemented contributions are:

- an integrated web interface for agricultural coverage planning and semantic mission route preparation;
- interactive controls for row spacing, waypoint spacing, rectangle alignment, square-up operations, and furrow orientation;
- optional OSM/OSRM path routing with honest snap warnings, plus local manual-zone validation and A* fall-back in the browser prototype;
- a shared waypoint export schema containing WGS84 latitude, longitude, ENU yaw, and planar quaternion fields; and
- a reproducible IEEE-style paper package with current UI screenshots, route metrics, tests, and compiled PDF output.

II. RELATED WORK

Coverage path planning is a core capability for robots that must observe or treat a complete area. Classical coverage work and surveys describe how back-and-forth patterns can reduce missed regions and unnecessary travel in structured environments [1], [2]. Agricultural vehicles add constraints that are less visible in indoor coverage: row spacing, implement width, turning behavior, service locations, headlands, and irregular field boundaries. Oksanen and Visala showed that field geometry has a direct effect on agricultural coverage efficiency [3]. More recent open-source work such as Fields2Cover provides reusable components for agricultural coverage planning and benchmarking [4].

Outdoor navigation between semantic targets is often framed as a geographic routing problem. OpenStreetMap-derived networks can be extracted and analyzed with tools such as OSMnx [5]. Grid or graph search can then combine geometric distance with semantic or terrain costs. A* remains a standard search method for this kind of discrete planning problem [6]. GeoZigzag Studio uses this family of methods in two ways: a simple local cost-aware route in the reproducible Python demo and a browser-side local A* fallback around manually drawn forbidden zones.

The output of such planners still needs to be executed by robot software. ROS2 provides middleware for distributed robot systems [7], and Navigation2 illustrates the execution side of mobile-robot navigation in ROS2 [8]. The work presented here does not replace local planning, localization, or control. Instead, it focuses on the pre-execution layer where an operator prepares route files, checks map context, and exports waypoints for later use.

III. DESIGN REQUIREMENTS

The tool was revised around four practical requirements. First, the coverage mode must expose agricultural parameters directly. The current interface therefore provides row spacing and waypoint spacing sliders, start-corner selection, furrow angle control, rectangle drawing, square-up operations, and North-South or East-West alignment actions. These controls are visible and interactive rather than hidden behind a script-only configuration.

Second, mission routing must keep the visit-order workflow. The mission page loads semantic GeoJSON points, shows them on the map, lets the user reorder or remove stops in a centered waypoint modal, and allows custom waypoints to be created by clicking the map. This preserves the operator-facing behavior of a geographic route planner while placing the route summary and generation button in a stable bottom-right map block.

Third, route feedback must distinguish normal success, warnings, and errors. The interface now uses green for generated routes, yellow/orange for generated routes with limitations such as OSM snapping, red for failed generation, and blue/neutral feedback for drawing modes. Verbose technical messages are placed inside a collapsed *Route details* panel so that the main interface remains usable during route preparation.

Fourth, the implementation must avoid unsupported claims. The current prototype does not download OpenStreetMap building footprints automatically and does not validate robot execution. Obstacles or buildings that should influence the route are represented as manual forbidden zones drawn by the user. The paper therefore describes OSM/OSRM as an optional routing mode and manual zones as a planning-time aid, not as a certified safety layer.

IV. SYSTEM OVERVIEW

The repository is organized around two execution paths. The first path is the browser application in `web/index.html`.

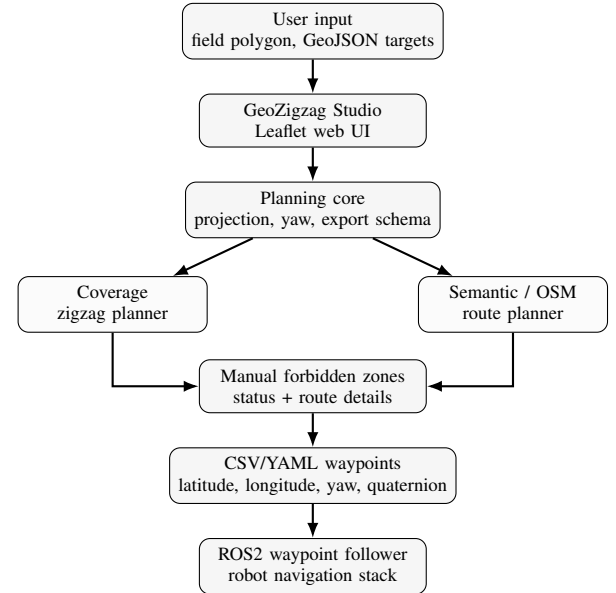


Fig. 1. Implemented workflow. GeoZigzag Studio prepares route files and metadata for later robot execution; it does not launch navigation itself.

It uses Leaflet [9] for GPS imagery and map views, stores manual forbidden zones in browser state, queries OSRM when optional OSM routing is selected, and generates downloadable CSV/YAML route files. The second path is the Python package `geozigzag`, which contains deterministic planning functions used by the command-line demo and unit tests.

Figure 1 summarizes the implemented workflow. User inputs include field polygons, rectangle corners, GeoJSON targets, custom waypoints, and manual forbidden zones. These are handled by the web UI and planning core. The planner then produces either coverage rows or semantic mission routes. Finally, the route is validated at the UI level, summarized for the operator, and exported as waypoint files. The last block is deliberately external: ROS2 waypoint following or robot navigation can consume the files, but it is not launched by the current application.

A. Coverage Interface

Figure 2 shows the latest coverage interface. The left panel exposes the field shape and geometry controls, while the map displays the generated zigzag route over GPS imagery. The current field can be represented as a rectangle or free polygon. In rectangle mode, corner markers can be dragged on the map or edited numerically. The rectangle can be squared, aligned North-South or East-West, aligned to the current furrow direction, reset, and applied or cancelled.

The furrow parameter panel provides row spacing, waypoint spacing, and furrow angle controls. Row spacing changes the number of swaths, whereas waypoint spacing changes the density of samples along each swath. The route summary is grouped with the generate button and export buttons on the map. This placement keeps the main route-generation controls

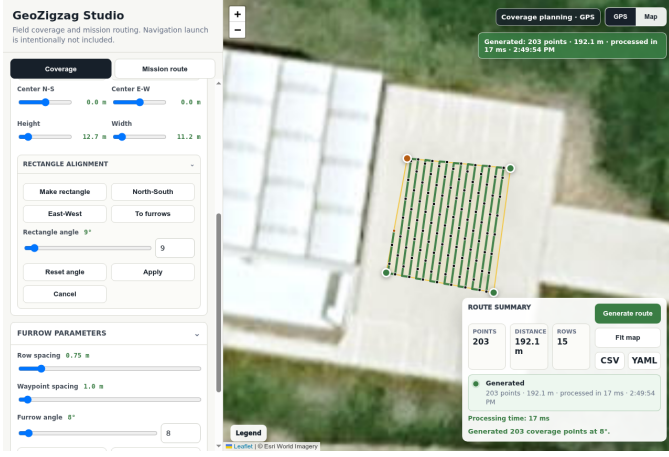


Fig. 2. Latest coverage-planning interface. The screenshot shows rectangle alignment controls, row-spacing and waypoint-spacing sliders, furrow-angle control, generated zigzag rows, route summary, and CSV/YAML export buttons.

available even when the left panel is scrolled to geometry settings.

B. Mission Route Interface

Figure 3 shows the mission-route interface. The route mode selector is placed on the right side of the map above the route summary. The waypoint manager opens as a centered modal rather than as a side panel that covers routing controls. Manual forbidden zones are listed in the left panel and are also drawn on the map. Each zone can be created, edited, cancelled, or deleted from the interface.

The screenshot uses optional OSM/OSRM routing in balanced mode. This mode tries to reach exact mission points while checking manually drawn forbidden zones. Because OSRM may snap input points to nearby routable roads or tracks, the route can be generated with warnings. The visible message remains short, while the detailed snap distances and fallback messages are kept in the collapsed *Route details* panel.

V. OPERATOR WORKFLOW

The intended workflow begins with visual route preparation rather than with a robot launch command. In coverage mode, the operator selects a field shape, checks the satellite or map view, adjusts the field corners or polygon vertices, and chooses the row spacing. The route is then generated explicitly with the *Generate route* button. This interaction is deliberate: editing geometry marks the route as pending, but it does not silently overwrite the previous preview or export files.

After generation, the operator inspects three levels of feedback. The map shows the spatial route and sampled waypoint markers. The route summary reports the number of points, approximate distance, and number of rows. The status block reports whether the generation succeeded, produced warnings, or failed. This separation is useful during field preparation because a syntactically valid waypoint file can still be wrong if the furrow angle, start corner, or field boundary is incorrect.

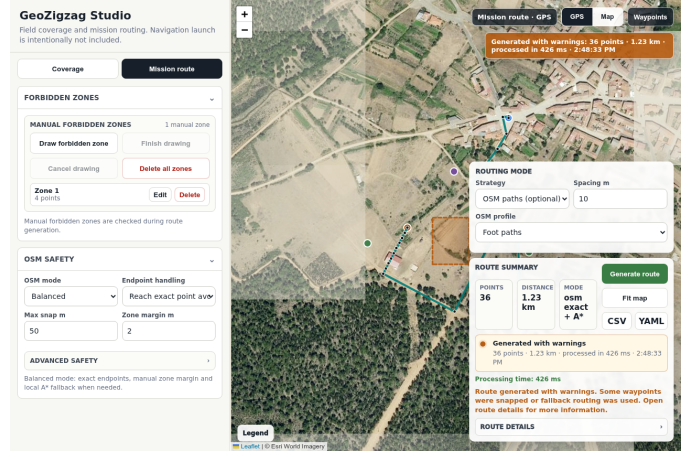


Fig. 3. Latest mission-route interface. Optional OSM/OSRM routing is selected with balanced manual-zone checks; the map shows a generated route, a manually drawn forbidden zone, status-coded warnings, and collapsed route details.

TABLE I
MAIN FILES AND ARTIFACTS IN THE REPRODUCIBLE PACKAGE.

Path	Role
web/index.html	Interactive Leaflet application
geozigzag/planning.py	Deterministic planning core
geozigzag/cli.py	Demo route generator
data/points.geojson	Semantic mission targets
outputs/*.csv	Inspectable waypoint exports
outputs/*.yaml	ROS-style waypoint exports
outputs/summary.json	Reproducible route metrics
paper/figures/*.png	Current UI screenshots

In mission mode, the workflow starts from existing GeoJSON targets or custom points added by clicking the map. The waypoint modal keeps the target list separate from the route-generation controls. The operator can reorder targets, remove points, or create named custom waypoints before asking for a route. The same route summary block is then used for direct, cost-aware, and optional OSM/OSRM modes, so the export workflow remains consistent across strategies.

Manual forbidden zones are introduced only when the user has a reason to mark a region as unsafe or invalid. The zone is drawn as a polygon, listed in the left panel, and included in the dynamic legend. It can be edited by returning the polygon to drawing mode and then applying or cancelling the edit. This behavior is intentionally simple: it gives the operator a transparent planning aid without implying that all real obstacles have been detected automatically.

The final step is export. The CSV file is convenient for inspection in a spreadsheet or plotting tool. The YAML file preserves the same coordinates and orientation fields in a structure that can be adapted to a ROS2 waypoint follower. The generated outputs are therefore useful both for human review and for later robot-side integration. Table I lists the main files produced or used by the reproducible package.

VI. PLANNING METHODS

A. Local Projection and Heading

The reproducible planning core uses a local tangent-plane approximation for short-distance geometry. For a WGS84 point (ϕ, λ) and origin (ϕ_0, λ_0) , the implementation computes

$$x = R \cos(\phi_0)(\lambda - \lambda_0), \quad y = R(\phi - \phi_0), \quad (1)$$

where angular values are in radians and R is the Earth radius used by the implementation. This approximation is sufficient for the tens or hundreds of metres used in the demonstration and avoids external numerical dependencies.

Each exported waypoint receives an ENU yaw angle computed from the following segment. The last waypoint reuses the previous heading. For compatibility with ROS-style waypoint consumers, yaw ψ is converted to a planar quaternion:

$$q_z = \sin(\psi/2), \quad q_w = \cos(\psi/2), \quad q_x = q_y = 0. \quad (2)$$

Both CSV and YAML exports include latitude, longitude, yaw, and quaternion fields. The shared export schema is important because it lets coverage routes and mission routes be inspected and consumed in the same downstream workflow.

B. Zigzag Coverage Planning

Coverage planning samples parallel sweep lines inside the selected field. For rectangular fields, the selected start corner defines the local origin and field axes. For free polygons, the planner intersects sweep lines with the polygon boundary and samples the valid row intervals. Consecutive rows alternate traversal direction, producing a boustrophedon pattern.

The current implementation separates row spacing from waypoint spacing. Row spacing controls the distance between neighboring swaths and therefore affects coverage density and path length. Waypoint spacing controls how frequently the continuous row is sampled for export. This distinction is useful for robot software: a user may want the same geometric path with fewer waypoints, or a denser route file without changing the agricultural row spacing.

Furrow orientation can be automatic or manual. When manual orientation is used, the selected angle is shown in degrees and can be aligned to the rectangle or set to common North-South and East-West values. The preview updates after the user presses the route-generation button, and the generated path is rendered on the map before export.

C. Semantic, OSM, and Manual-Zone Routing

Mission routing operates over an ordered list of GeoJSON features and custom map points. Direct mode interpolates between targets at the requested spacing. The reproducible Python cost-aware mode builds a small local grid, assigns costs according to land-cover labels, and runs A* between consecutive targets. This costmap is intentionally lightweight and deterministic; it is not a validated traversability model.

The browser UI also includes optional OSM/OSRM routing. It queries OSRM segment-by-segment and can request foot-path or driving profiles. The mode is useful for previewing

TABLE II
REGENERATED ROUTE METRICS FROM THE REPOSITORY DEMO.

Route	Points	Distance (m)	Rows
Coverage zigzag	224	205.8	16
Mission direct	54	518.2	–
Mission cost-aware	83	542.5	–

TABLE III
COVERAGE SENSITIVITY REGENERATED FROM THE PYTHON CORE.

Row (m)	Point (m)	Rows	Points
0.50	1.00	23	322
0.75	1.00	16	224
1.00	1.00	12	168
1.50	1.00	9	133
0.75	0.50	16	430
0.75	2.00	16	128

routes over mapped paths, but it depends on an external service and can snap target points to nearby routable geometry. For this reason the paper treats OSM/OSRM as optional and the UI reports snap distances as warnings.

Manual forbidden zones replace the previous idea of automatically downloading building footprints. A user draws a polygon around a house, shed, or area that should not be crossed. In balanced or strict OSM modes, the browser checks route segments against these zones and can use a local A* fallback around them. This is a planning-time aid for route preparation; it is not equivalent to onboard obstacle avoidance during robot execution.

VII. IMPLEMENTATION AND REPRODUCIBILITY

The repository keeps the implementation intentionally small. The main files are `web/index.html` for the browser app, `geozigzag/planning.py` for the deterministic planning functions, `geozigzag/cli.py` for the demo command, `tests/test_planning.py` for unit tests, and `paper/main.tex` for the manuscript. Generated outputs are written to `outputs/`, while paper screenshots are stored under `paper/figures/`.

The command-line demo is:

```
python3 -m geozigzag.cli --out outputs
```

It regenerates one coverage mission and two semantic missions from `data/points.geojson`. The semantic mission order is `water_1`, `arbustivo_2`, and `water_2`. Table II reports the stable metrics regenerated from this command.

The coverage route is denser because it represents intra-field traversal. The semantic routes connect sparse targets separated by hundreds of metres. The cost-aware route contains more points and a longer path than direct interpolation because it follows the grid-search result. These values are not robot-performance metrics; they verify that the route-generation and export pipeline can be reproduced.

Table III illustrates why the interface exposes row spacing and waypoint spacing separately. Changing row spacing modifies the number of coverage rows and total distance. Changing waypoint spacing at a fixed row spacing changes the number of

TABLE IV
IMPLEMENTED INTERFACE FEATURES DESCRIBED IN THE PAPER.

Feature	Current implementation
Coverage generation	Rectangle and polygon modes
Spacing control	Row and waypoint sliders
Furrow orientation	Degree slider and quick actions
Rectangle alignment	Square, N-S, E-W, apply/cancel
Mission targets	GeoJSON points and custom clicks
Waypoint manager	Centered modal with reordering
Route modes	Direct, cost-aware, optional OSM/OSRM
Forbidden zones	Manual draw, edit, delete, validate
Route feedback	Status colors and collapsed details
Export	CSV/YAML with yaw and quaternion

exported points while preserving the same geometric coverage path.

The current validation includes unit tests for default zigzag generation, polygon coverage, yaw and quaternion conversion, semantic route generation, export schemas, summary JSON shape, and invalid spacing rejection. The paper was compiled with IEEEtran conference mode on A4 paper and kept within the six-page regular-paper limit specified by the conference instructions. The compiled PDF should still be checked with IEEE PDF eXpress for the final camera-ready submission if the paper is accepted.

VIII. IMPLEMENTATION AUDIT

Because the paper is intended to describe a changing web prototype, the claims were checked against the current repository rather than against planned features. Table IV summarizes the main interface elements and how they are represented in the implementation. This audit is included to make the scope explicit: the system prepares and exports routes, while execution remains external.

Two implementation choices are worth noting. First, automatic OpenStreetMap building download was removed from the user-facing workflow because it made the route-generation path slower and less predictable. The current tool keeps manual forbidden zones instead; this better matches an operator-driven route review process and avoids presenting incomplete map data as a reliable obstacle source. Second, route generation is now explicitly triggered by the *Generate route* button. Editing a field, waypoint list, or forbidden zone marks the route as pending, but it does not silently regenerate until the user asks for a new preview. This behavior makes screenshots and route exports more traceable because each exported file corresponds to a visible generation action.

The screenshots were regenerated from the local web application after these UI changes. Figure 2 therefore shows the current coverage controls rather than an earlier panel layout, and Fig. 3 shows the current route summary, routing mode block, manual-zone list, and collapsed route details. This prevents the manuscript from documenting stale menus such as the removed building-cache controls.

IX. DISCUSSION

The main value of GeoZigzag Studio is workflow continuity. The same interface lets an operator select or edit a field,

tune coverage parameters, switch to a mission route, compare routing modes, draw forbidden zones, inspect status messages, and export route files. This reduces the risk that paper figures, operator screenshots, and route files describe different versions of the software.

The UI changes made during the latest revision are also important for practical use. The route summary and generate button are grouped in the map’s bottom right corner, route mode remains visible above them, and waypoint management is handled through a modal. These layout decisions prevent the map controls, route summary, and waypoint list from overlapping. The dynamic legend shows only visible layers, which helps distinguish coverage rows, OSM routes, waypoints, and manual forbidden zones.

The OSM/OSRM mode should be interpreted cautiously. It provides a useful map path preview, but the public routing service may fail, be slow, or snap targets away from their exact geographic coordinates. The application therefore keeps direct and local cost-aware routes available and reports OSM limitations as warnings rather than hiding them. This design is preferable to presenting a snapped route as if it were always safe for a robot to execute.

X. LIMITATIONS AND FUTURE WORK

The current system prepares routes; it does not execute them. It does not perform localization, closed-loop path tracking, online obstacle avoidance, or failure recovery. It also does not launch Gazebo, ROS2 navigation, or the robot’s crop-follow stack. Those capabilities are natural next integration steps, but they should be evaluated separately and not implied by the present software artifact.

The forbidden-zone mechanism is deliberately manual. It lets the user encode visible buildings or unsafe regions that matter for a route preview, but it is not a substitute for onboard perception or certified safety monitoring. Future versions could ingest surveyed obstacles, crop-row maps, orthomosaic-derived cost layers, or offline OpenStreetMap extracts. Such additions should preserve the current route-detail log so operators can understand why a route was accepted, warned, or rejected.

Future robot validation should measure planned-versus-executed error on an agricultural platform. For coverage missions, relevant metrics include cross-track error, missed area, overlap, number of turns, and execution time. For semantic routes, useful metrics include arrival error at each target, intervention count, distance travelled, and local planner recoveries. These experiments would turn the current reproducible planning tool into a broader navigation evaluation pipeline.

XI. THREATS TO VALIDITY

The reported metrics are generated from a small deterministic dataset. They are useful for regression testing and reproducibility, but they do not represent statistical performance over many farms, crops, or terrain types. The coverage example is a compact field patch, and the semantic mission uses a fixed order of three targets. Larger and more irregular

scenarios may reveal usability or performance issues that are not visible in this demonstration.

The local projection used by the Python core is appropriate for short routes, but it is still an approximation. For large farms or missions spanning several kilometres, a more rigorous geodetic treatment would be preferable. The browser view also depends on external imagery and tile providers; the visual context seen by the operator can change if those services update their data.

The optional OSM/OSRM mode introduces another source of variability. The public routing service may return different paths as map data changes, may snap waypoints to roads or tracks, or may be temporarily unavailable. For this reason, the paper does not use OSRM output as the deterministic metric source. The reproducible metrics come from the local Python planner, while the browser screenshots document the current UI behavior.

Manual forbidden zones also depend on operator judgment. They are valuable when the operator can identify a house, shed, or unsafe region from map context, but they are not automatically complete. Missing a forbidden zone in the planning interface does not imply that the physical robot can safely traverse the area. Future work should compare manually drawn zones with surveyed obstacles and onboard perception outputs.

XII. CONCLUSION

GeoZigzag Studio integrates coverage planning and semantic georouting into a single web-based route-preparation workflow. The implemented application supports zigzag furrow generation, orientation controls, semantic mission routes, optional OSM/OSRM preview, manual forbidden zones, route-status feedback, technical route logs, and CSV/YAML waypoint export. The paper package matches the current implementation, regenerates figures from the latest UI, and reports only metrics that can be reproduced from the repository. The result is a practical basis for future agricultural robot experiments and for subsequent ROS2/Gazebo integration.

REFERENCES

- [1] H. Choset, "Coverage for robotics: A survey of recent results," *Annals of Mathematics and Artificial Intelligence*, vol. 31, no. 1, pp. 113–126, 2001.
- [2] E. Galceran and M. Carreras, "A survey on coverage path planning for robotics," *Robotics and Autonomous Systems*, vol. 61, no. 12, pp. 1258–1276, 2013.
- [3] T. Oksanen and A. Visala, "Coverage path planning algorithms for agricultural field machines," *Journal of Field Robotics*, vol. 26, no. 8, pp. 651–668, 2009.
- [4] G. Mier, R. Valente, J. Valente, A. Barrientos, and J. del Cerro, "Fields2Cover: An open-source coverage path planning library for unmanned agricultural vehicles," 2023.
- [5] G. Boeing, "OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks," *Computers, Environment and Urban Systems*, vol. 65, pp. 126–139, 2017.
- [6] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [7] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, 2022.

- [8] S. Macenski and F. Martin, "The Marathon 2: A navigation system," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020.
- [9] V. Agafonkin, "Leaflet: An open-source JavaScript library for interactive maps." [Online]. Available: <https://leafletjs.com/>